Multi-Level Caching

an in-process L1 in front of a shared L2

Watch the 4-min video

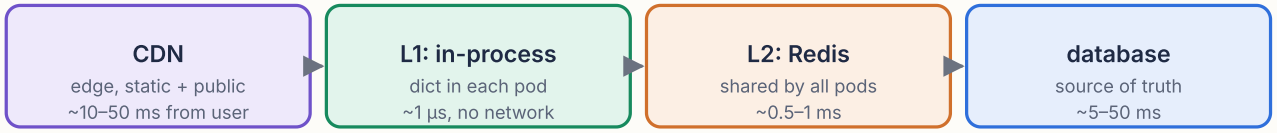
In one line: multi-level caching stacks a tiny, microsecond in-process L1 (near cache) in each pod in front of a shared L2 like Redis and the database; reads check L1, then L2, then the database and fill back; the hard part is invalidating hundreds of L1 copies, handled with short L1 TTLs and pub/sub broadcasts, with an explicit staleness bound.

1 Half a million Redis calls a second

Your Redis cache is fast: one millisecond per lookup. But your service does 50 lookups per request and runs at 10,000 requests per second. That's **500,000 Redis calls per second**, plus network time on every one: 50 ms of waiting per request if done one by one. What if the hottest data lived inside each process?

2 The levels

Multi-level caching stacks caches from fastest and smallest to slowest and largest. **L1** is an in-process cache inside each application instance: a dictionary in memory, lookups in microseconds, no network. **L2** is a shared distributed cache like Redis: milliseconds, bigger, shared by all instances. Behind them, the database; in front of everything, the CDN. Like CPU caches, each level catches what the faster one misses.



fastest and smallest on the left, slowest and largest on the right

Figure 1. The cache levels, from the edge to the source of truth.

3 The read path

A read checks L1 first. On a miss, it checks L2. On another miss, it reads the database, then fills L2 and L1 on the way back. Most hot reads stop at L1. L1 is often called a **near cache**, because it sits right next to the code. In Python, `cachetools.TTLCache` gives you one; in Java, Caffeine.



most hot reads stop at step 1

Figure 2. A two-level read: L1, then L2, then the database, filling back on the way.

4 The hard part: invalidating every L1

When data changes you can delete it from Redis, but you can't easily reach the L1 caches inside 200 pods. Common answers:

- **Very short L1 TTLs**, a few seconds, so L1 copies expire on their own.
- **Broadcast invalidations** with Redis pub/sub or a message bus: every pod subscribes and deletes the key from its local cache.
- **Redis client-side caching** (`CLIENT TRACKING`, Redis 6+): the server remembers which keys each client read and pushes invalidations automatically.

WORKED EXAMPLE: FEATURE FLAGS

Flags are read on almost every request. Each pod keeps them in an in-process TTL cache for **5 seconds**, with Redis as L2 holding them for **5 minutes**. When someone flips a flag, the admin service updates the database, deletes the Redis key and publishes a `flag-changed` event. Pods drop their L1 copy within milliseconds; a pod that misses the event (pub/sub is fire-and-forget) still picks up the change within 5 seconds, because of the TTL.

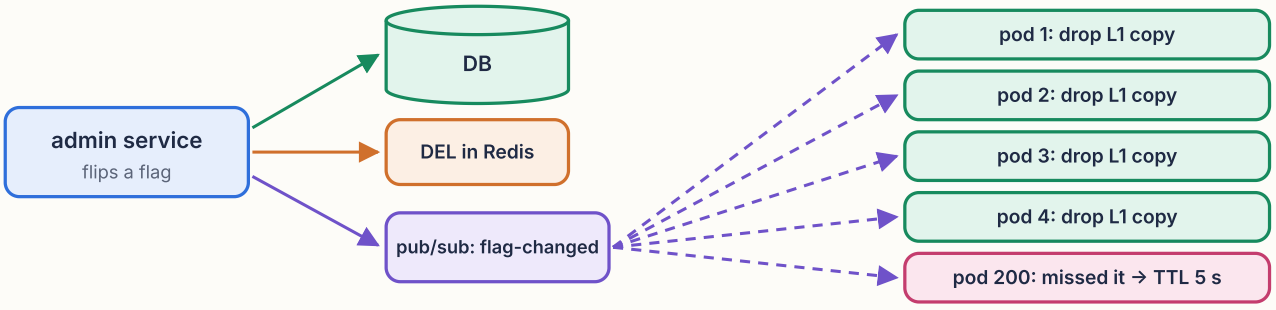


Figure 3. Invalidating L1 across pods: delete in Redis, broadcast an event, and rely on the short L1 TTL for any pod that missed it.

5 Risks of in-process caches across many pods

- **Inconsistency**: different pods can hold different versions, so two requests from the same user might see different values.
- **Memory multiplies**: every pod stores its own copy; 200 pods × 500 MB = 100 GB.
- **Cold starts**: new or restarted pods begin empty and add load on L2.
- **Invalidation fan-out** to hundreds of pods adds moving parts.

So use L1 for **small, hot, read-mostly** data that tolerates a few seconds of staleness: configuration, feature flags, reference data, hot keys (lesson 26). Keep user-specific or rapidly changing data in L2 only, and always state the staleness bound.

COMMON MISTAKES

- An L1 with no TTL and no size limit: stale forever and a memory leak.
- Caching per-user data in L1, so memory explodes and users see inconsistent results across pods.
- Relying only on pub/sub for invalidation: messages can be missed.

6 Key terms

L1 / near cache: in-process cache in each instance

L2: shared distributed cache (Redis)

Fill back: populate faster levels on the way out

Client-side caching: Redis tracks keys and pushes invalidations

Invalidation broadcast: pub/sub message telling every pod to drop a key

Staleness bound: the maximum age you promise

7 Interview questions

Q1. What are the risks of in-process caches across many pods? BASIC

Pods can serve different, inconsistent versions, memory is duplicated per pod, new pods start cold, and invalidating every pod's copy is hard.

Q2. How do you keep levels in sync? BASIC

Short L1 TTLs, invalidation broadcasts through pub/sub, and deleting from L2 on writes, accepting a small, explicit staleness window.

Q3. Each request reads 50 config values from Redis. How do you cut that? SCENARIO

Put an L1 TTL cache (a few seconds) in each process for config, or load the whole config periodically into memory; also batch remaining Redis reads with MGET or pipelining.

Q4. Which data belongs in L1? SCENARIO

Small, hot, read-mostly data that tolerates seconds of staleness: feature flags, configuration, reference tables, and the hottest keys. Not carts, balances or per-user data.

Q5. Why isn't pub/sub alone enough to invalidate L1 caches? TRICKY

Redis pub/sub is fire-and-forget: a pod that is disconnected or restarting misses the message and keeps the stale copy. A short TTL bounds that; streams or versioned keys make it more reliable.

Q6. How does Redis client-side caching work? TRICKY

With CLIENT TRACKING the server remembers which keys a client has read and sends it an invalidation message when one of them changes, so the client's local copy can be dropped.

Next: SD-F-019 · Caching in Python Applications backendroadmap.com